Chapter 5

ARRAYS AND FUNCTIONS

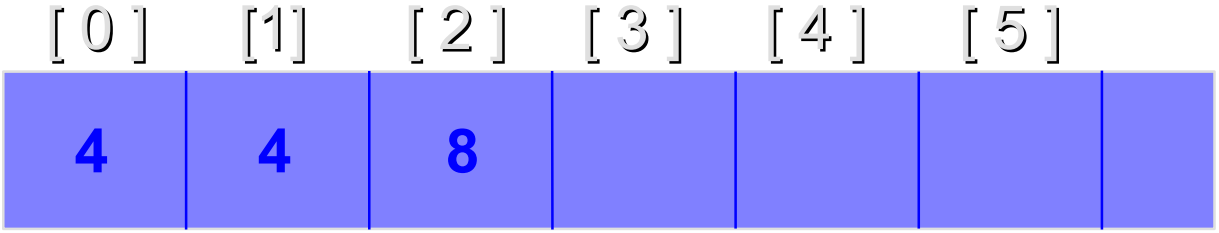
Outline

- 1. Arrays
- 2. Multidimensional Arrays
- 3. Function and parameter declarations
- 4. Pass-by-value
- 5. Variable scope
- 6. Variable storage classes
- 7. Pass-by-reference
- 8. Recursion
- 9. Passing arrays to functions

1. ARRAYS

- An *array* is an advanced data type that contains a set of data represented by a single variable name.
- An element is an individual piece of data contained in an array.
- The following figure shows an integer array called *c*.

  c[0] = 4; c[1] = 4, c[2] = 8, etc.



Array Declaration

- **The syntax for declaring an array is**

  type name[elements];

- **Array names follow the same naming conventions as variable names and other identifiers.**

- **Example:**

```
int arMyArray[3];
char arStudentGrade[5];
```

- **The first declaration tells the compiler to reserve 3 elements for integer array *arMyArray*.**

Subscript

- **The numbering of elements within an array starts with an index number of 0.**

- **An *index number* is an element's numeric position within an array. It is also called *a subsript*.**

- **<u>Example</u>:**

***StudentGrade[0]* refers to the 1st element in the *StudentGrade* array.**
***StudentGrade[1]* refers to the 2nd element in the *StudentGrade* array.**
***StudentGrade[2]* refers to the 3rd element in the *StudentGrade* array.**
***StudentGrade[3]* refers to the 4th element in the *StudentGrade* array.**
***StudentGrade[4]* refers to the 5fth element in the *StudentGrade* array.**

An example of array

Example 5.1.1

```
#include <iostream>
using namespace std;
int main() {
    char arStudentGrade[5] = {'A', 'B', 'C', 'D', 'F'};
    for (int i = 0; i < 5; i++)
       cout << arStudentGrade[i] << endl;
  return 0;
}
```

The output is:

```
A
B
C
D
F
```

Example 5.1.2

```
// Compute the sum of the elements of the array
#include <iostream>
using namespace std;
int main()
{
   const int arraySize = 12;
   int a[arraySize] = { 1, 3, 5, 4, 7, 2, 99, 16, 45, 67, 89, 45 };
   int total = 0;
   for ( int i = 0; i < arraySize; i++ )
      total += a[ i ];
   cout << "Total of array element values is " << total << endl;
   return 0 ;
}
```

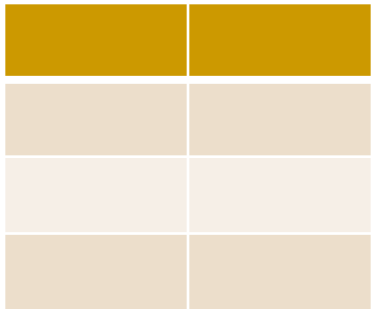
The output of the above program is as follows :

```
Total of array element values is  383
```

2. Multi-Dimensional Arrays

- **C++ allows arrays of any type, including arrays of arrays. With two bracket pairs we obtain a *two-dimensional* array.**
- **The idea can be iterated to obtain arrays of higher dimension. With each bracket pair we add another dimension.**

- **Some examples of array declarations** int t[4][2];



```
int a[1000];        // a one-dimensional array
int b[3][5];        // a two-dimensional array
int c[7][9][2];     // a three-dimensional array
```

In these above examples, *b* has 3 × 5 elements, and *c* has 7 × 9 × 2 elements.

A two-dimensional array

- **Starting at the base address of the array, all the array elements are stored contiguously in memory.**
- **For the array *b*, we can think of the array elements arranged as follows:**

```
int b[3][5];
```

	col 1	col2	col3	col4	col5
row 1	b[0][0]	b[0][1]	b[0][2]	b[0][3]	b[0][4]
row 2	b[1][0]	b[1][1]	b[1][2]	b[1][3]	b[1][4]
row 3	b[2][0]	b[2][1]	b[2][2]	b[2][3]	b[2][4]

Example 5.2.1 This program checks if a matrix is symmetric or not.

```
#include<iostream>
using namespace std;
const int N = 3;
void main( )
{
   int i, j;
   int a[N][N];
   bool symmetr = true;
   for (i=0; i<N; ++i)
      for (j=0; j<N; ++j)
          cin >> a[i][j];
   for(i= 0; i<N; i++){
         for (j = 0; j < N; j++)
            if(a[i][j] != a[j][i]) {
               symmetr = false;
               break;
            }
         if(!symmetr)
            break;
   }
   if(symmetr)
     cout<<"\nThe matrix is symmetric“
         << endl;
   else
     cout<<"\nThe matrix is not symmetric“
         << endl;
   return 0;
}
```

0,0	0,1	0,2
1,0	1,1	1,2
2,0	2,1	2,2

if(!symmetr) means **if(symmetry == false)**

3. Function and parameter declarations

- **User-defined program units are called *subprograms*. In C++ all subprograms are referred to as functions.**

- **<u>Defining a Function</u>**

  The lines that compose a function within a C++ program are called a *function definition*.

The syntax for declaring & defining a function:

```
data_type  name_of_function  (parameters) {
        statements;
}
```

To call a function

```
data_type  result = name_of_function (param1, param2, …)
```

- **A function definition consists of four parts:**

- A reserved word indicating the data type of the function's return value.
- The function name
- Any parameters required by the function, contained within (and).
- The function's statements enclosed in curly braces { }.

- **Example 1:**

```
void FindMax(int x, int y) {
   int maxnum;
   if ( x > = y)
      maxnum = x;
   else
       maxnum = y;
   cout << "\n The maximum of the two numbers is " << maxnum << endl;
   return;
}
```

How to call functions

- **You designate a data type for function since it will return a value from a function after it executes.**
- **Variable names that will be used in the function header line are called *formal parameters*.**
- **To execute a function, you must invoke, or call, it from the *main()* function.**
- **The values or variables that you place within the parentheses of a function call statement are called *actual parameters*.**

- **Example:**

```
int firstNum = 5;
int secNum = 8;
findMax(firstNum, secNum);
```

Function Prototypes

- **A *function prototype* declares to the compiler that you intend to use a function later in the program.**
- **If you try to call a function at any point in the program prior to its function prototype or function definition, you will receive an *error* at compile time.**

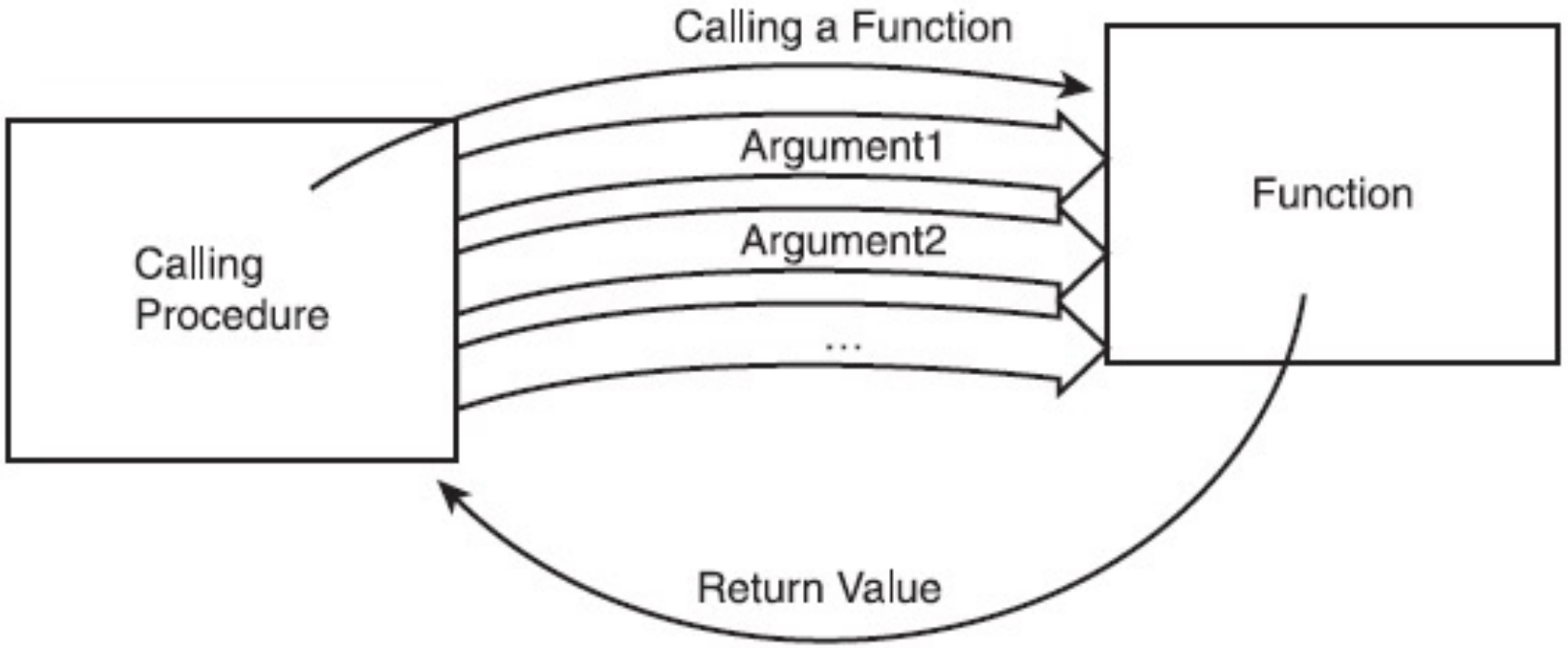
Example 5.3.1

```
// Finding the maximum of three integers
#include <iostream>
using namespace std;
int maximum(int, int, int);   // function prototype (forward declaration)
int main()
{
   int a, b, c;
   cout << "Enter three integers: ";
   cin >> a >> b >> c;
   cout << "Maximum is: " << maximum (a, b, c) << endl;
   return 0;
}
-
```

```
// Function maximum definition
// x, y and z are parameters to the maximum function definition
int maximum( int x, int y, int z)
{
   int max = x;
   if ( y > max )
      max = y;
   if ( z > max )
      max = z;
   return max;
}
```

The output of the above program:

```
Enter three integers: 22  85  17
Maximum is:  85
```



Conceptual diagram of the process of calling a function

4. Pass by Value

- **If a variable is one of the actual parameters in a function call, the called function receives a *copy* of the values stored in the variable.**

- **After the values are passed to the called function, control is transferred to the called function.**

- **Example: The statement *findMax(firstnum, secnum)*; calls the function *findMax* and causes the *values* currently residing in the variables *firstnum* and *secnum* to be passed to *findMax* function.**

- **The method of passing values to a called function is called *pass by value*.**

RETURNING VALUES

- **To actually return a value to a variable, you must include the *return* statement within the called function.**

- **The syntax for the *return* statement is either**

```
return result;
```

or

```
return(result);
```

- **Values passes back and forth between functions must be of the same data type.**

5. VARIABLE SCOPE

- **Scope refers to where in your program a declared variable or constant is allowed used.**
- ***Global scope* refers to variables declared outside of any functions or classes and that are available to all parts of your program.**
- ***Local scope* refers to a variable declared inside a function and that is available only within the function in which it is declared.**

Example 5.5.1

```
#include <iostream>
using namespace std;
int x;    // create a global variable named x
void valfun();   // function prototype (declaration)
int main()
{
  int y;    // create a local variable named y
  x = 10; // store a value into the global variable
  y = 20;   // store a value into the local variable
  cout << "From main(): x = " << x << endl;
  cout << "From main(): y =  " << y << endl;
  valfun();      // call the function valfun
  cout << "\nFrom main() again: x = " << x << endl;
  cout << "From main() again: y = " << y << endl;
  return 0;
}
```

```
void valfun() {
  int y;  // create a second local variable named y
  y = 30; // this only affects this local variable's value
  cout << "\nFrom valfun(): x = " << x << endl;
  cout << "\nFrom valfun(): y = " << y << endl;
  x = 40;    // this changes x for both functions
  return;
}
```

The output of the above program:

```
From main(): x = 10
From main(): y = 20

From valfun(): x = 10
From valfun(): y = 30

From main() again: x = 40
From main() again: y = 20
```

6. VARIABLE STORAGE CLASS

- **The lifetime of a variable is referred to as the *storage duration*, or *storage class*.**

- **Four available storage classes: *auto, static, extern* and *register*.**

- **If one of these class names is used, it must be placed before the variable's data type in a declaration statement.**

- **Examples:**

```
auto int num;
static int miles;
register int dist;
extern float price;
extern float yld;
```

Local Variable Storage Classes

- **Local variables can only be members of the *auto, static*, or *register* storage classes.**

- **Default: *auto* class.**

Automatic Variables

- **The term *auto* is short for automatic.**

- ***Automatic storage duration* refers to variables that exist only during the lifetime of the command block (such as a function) that contains them.**

Example 5.6.1

```
#include <iostream>
using namespace std;
void testauto();    // function prototype
int main(){
  int count;          // count is a local auto variable
  for(count = 1; count <= 3; count++)
      testauto();
  return 0;
}
void testauto(){
  int num = 0;     // num is a local auto variable
  cout << "The value of the automatic variable num is "
          << num << endl;
  num++;
  return;
}
```

The output of the above program:

The value of the automatic variable num is 0

The value of the automatic variable num is 0

The value of the automatic variable num is 0

Static local variables

- **In some applications, we want a function to remember values between function calls. This is the purpose of the *static* storage class.**
- **A local *static* variable is not created and destroyed each time the function declaring the static variable is called.**
- **Once created, local static variables *remain in existence* for the life of the program.**

Example 5.6.2

```
#include <iostream>
using namespace std;
int funct(int);    // function prototype
int main()
{
  int count, value;      // count is a local auto variable
  for(count = 1; count <= 10; count++)
     value = funct(count);
     cout << count << '\t' << value << endl;
  return 0;
}

int funct( int x)
{
  int sum = 100;     // sum is a local auto variable
  sum += x;
  return sum;
}
```

The output of the above program:

1	101
2	102
3	103
4	104
5	105
6	106
7	107
8	108
9	109
10	110

Note: The effect of increasing *sum* in *funct()*, before the function's *return* statement, is lost when control is returned to *main()*.

Local static variable

A local *static* variable is not created and destroyed each time the function declaring the static variable is called. Once created, local static variables *remain in existence* for the life of the program.

Example 5.6.3

```
#include <iostream>
using namespace std;
int funct( int);    // function prototype
int main()
{
  int count, value;          // count is a local auto variable
  for(count = 1; count <= 10; count++)
     value = funct( count);
     cout << count << '\t' << value << endl;
  return 0;
}
int funct( int x)
{
  static int sum = 100;     // sum is a local static variable
  sum += x;
  return sum;
}
```

The output of the above program:

```
1       101
2       103
3       106
4       110
5       115
6       121
7       128
8       136
9       145
10      155
```

Note:

1.The initialization of *static* variables is done only once when the program is first compiled. At compile time, the variable is created and any initialization value is placed in it.

2. All static variables are set to zero when no explicit initialization is given.

Register Variables

- ***Register variables*** **have the same time duration as automatic variables.**
- **Register variables are stored in CPU's internal registers rather than in memory.**
- **Examples:**

```
register int  time;
register double difference;
```

Global Variable

- Global variables are created by definition statements external to a function.
- Once a global variable is created, it exists until the program in which it is declared is finished executing.

7. PASS BY REFERENCE

- **Reference Parameters**

  Two ways to invoke functions in many programming languages are:

  - **call by value**
  - **call by reference**

- **When an argument is passed *call by value*, a copy of the argument's value is made and passed to the called function. Changes to the copy do not affect the original variable's value in the caller.**

- **With *call-by-reference*, the caller gives the called function the ability to access the caller's data directly, and to modify that data if the called function chooses so.**
- **To indicate that the function parameter is passed-by-reference, simply follow the parameter's type in the function prototype of function header by *an ampersand* (&).**

- **For example, the declaration**

  int& count

  in the function header means "*count* is a reference parameter to an *int*".

Example 5.7.1

```
#include <iostream>
using namespace std;
int squareByValue( int );
void squareByReference( int & );
int main()
{
   int x = 2, z = 4;
   cout << "x = " << x << " before squareByValue\n"
        << "Value returned by squareByValue: "
        << squareByValue( x ) << endl
        << "x = " << x << " after squareByValue\n" << endl;
   cout << "z = " << z << " before squareByReference" << endl;
   squareByReference( z );
   cout << "z = " << z << " after squareByReference" << endl;
   return 0;
}
```

```
int squareByValue( int a )
{
   return a *= a;   // caller's argument not modified
}
void squareByReference(int &cRef )
{
   cRef *= cRef;    // caller's argument modified
}
```

The output of the above program:

x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByReference

z = 4 before squareByReference
z = 16 after squareByReference

8. RECURSION

- In C++, it's possible for a function to call itself. Functions that do so are called *seft-referential* or *recursive functions*.
- Example: To compute factorial of an integer

  1! = 1

  n! = n*(n-1)!

Example 5.8.1

```
#include <iostream>
#include <iomanip>
using namespace std;
int factorial( int );
int main() {
    for ( int i = 0; i <= 10; i++ )
        cout << setw( 2 ) << i << "! = " << factorial( i ) << endl;
    return 0;
}
```

```
// Recursive definition of function factorial
int factorial( int number )
{
   if (number == 0)  // base case
      return 1;
   else                    // recursive case
      return number * factorial( number - 1 );
}
```

The output of the above program:

```
 0! = 1
 1! = 1
 2! = 2
 3! = 6
 4! = 24
 5! = 120
 6! = 720
 7! = 5040
 8! = 40320
 9! = 362880
10! = 3628800
```

9. PASSING ARRAYS TO FUNCTIONS

- **To pass an array to a function, specify the name of the array without any brackets. For example, if array *hourlyTemperature* has been declared as**

  int hourlyTemperature[24];

- **The function call statement**

  modifyArray(hourlyTemperature, size);

  passes the array *hourlyTemperature* and its size to function *modifyArray*.

- **For the function to receive an array through a function call, the function's parameter list must specify that an array will be received.**

For example, the function header for function *modifyArray* might be written as

```
void modifyArray(int b[], int arraySize)
```

Notice that the size of the array is not required between the array brackets.

Example 5.9.1

```
#include<iostream>
using namespace std;
int linearSearch( int [], int, int);
void main()
{
   const int arraySize = 100;
   int a[arraySize], searchkey, element;
```

```
   for (int x = 0; x < arraySize, x++)   // create some data
         a[x] = 2*x;
   cout<< "Enter integer search key: "<< endl;
   cin >> searchKey;
   element = linearSearch(a, searchKey, arraySize);
   if(element != -1)
     cout<<"Found value in element "<< element << endl;
   else
     cout<< "Value not found " << endl;
}
int linearSearch(int array[], int key, int sizeofArray)
{
    for(int n = 0; n< sizeofArray; n++)
      if (array[n] = = key)
        return n;
    return –1;
}
```